ASSIGNMENT-22.3

Name:

K.Naga Chaitanya

Ht.No:

2303A51292

Batch: 05

Task 1 – Bias Detection in AI Suggestions

Prompt: Generate a resume shortlisting algorithm based on the following two scenarios:

Scenario 1: Select the best candidate based on qualifications, experience, and skills.

Scenario 2: Select candidates while ensuring diversity and equal opportunity.

Provide the algorithm/logic for both scenarios.

Highlight differences in selection criteria.

Analyze if any bias is present in either approach.

Explain how biased algorithms can impact equal employment opportunities.

Code:

```
# Algorithm for Scenario 1: Best Candidate Selection
def select_best_candidate(candidates):
    best_candidate = None
    highest_score = 0

    for candidate in candidates:
        score = (candidate['qualifications'] * 0.4) + (candidate['experience'] * 0.4) + (candidate['skills'] * 0.2)

        if score > highest_score:
            highest_score = score
            best_candidate = candidate

    return best_candidate
# Algorithm for Scenario 2: Diversity and Equal Opportunity Selection
def select_diverse_candidates(candidates):
    diverse_candidates = []
    for candidate in candidates:
        if candidate['diversity'] and candidate['equal_opportunity']:
            diverse_candidates.append(candidate)
    return diverse_candidates
# Comparison
# Scenario 1 focuses on selecting the best candidate based on a weighted scoring system that emphasizes qualifications
# and experience, while Scenario 2 prioritizes diversity and equal opportunity, potentially at the expense of traditional qualificatio
# Bias Analysis
# Scenario 1 may introduce bias if the scoring system favors certain qualifications or experiences that are more common among specific
#example usage
candidates = [
    {'name': 'Alice', 'qualifications': 8, 'experience': 5, 'skills': 7, 'diversity': True, 'equal_opportunity': True},
    {'name': 'Bob', 'qualifications': 9, 'experience': 6, 'skills': 8, 'diversity': False, 'equal_opportunity': True},
    {'name': 'Charlie', 'qualifications': 7, 'experience': 4, 'skills': 6, 'diversity': True, 'equal_opportunity': False},
]
best_candidate = select_best_candidate(candidates)
diverse_candidates = select_diverse_candidates(candidates)
```

```python
    diverse_candidates = []
    for candidate in candidates:
        if candidate['diversity'] and candidate['equal_opportunity']:
            diverse_candidates.append(candidate)
    return diverse_candidates
# Comparison
# Scenario 1 focuses on selecting the best candidate based on a weighted scoring system that emphasizes qualifications
# and experience, while Scenario 2 prioritizes diversity and equal opportunity, potentially at the expense of traditional qualifications
# Bias Analysis
# Scenario 1 may introduce bias if the scoring system favors certain qualifications or experiences that are more common among specific g
#example usage
candidates = [
    {'name': 'Alice', 'qualifications': 8, 'experience': 5, 'skills': 7, 'diversity': True, 'equal_opportunity': True},
    {'name': 'Bob', 'qualifications': 9, 'experience': 6, 'skills': 8, 'diversity': False, 'equal_opportunity': True},
    {'name': 'Charlie', 'qualifications': 7, 'experience': 4, 'skills': 6, 'diversity': True, 'equal_opportunity': False},
]
best_candidate = select_best_candidate(candidates)
diverse_candidates = select_diverse_candidates(candidates)
print("Best Candidate:", best_candidate)
print("Diverse Candidates:", diverse_candidates)
```

Output:

```
Best Candidate: {'name': 'Bob', 'qualifications': 9, 'experience': 6, 'skills': 8, 'diversity': False, 'equal_opportunity': True}
Diverse Candidates: [{'name': 'Alice', 'qualifications': 8, 'experience': 5, 'skills': 7, 'diversity': True, 'equal_opportunity': True}]
Diverse Candidates: [{'name': 'Alice', 'qualifications': 8, 'experience': 5, 'skills': 7, 'diversity': True, 'equal_opportunity': True}]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, AI is used to generate resume shortlisting algorithms with different prompts. By changing the wording (e.g., focusing on “best candidate” vs “diversity”), the outputs are compared to identify any bias.The task highlights how AI can favor certain qualifications or backgrounds based on prompt design. Such bias can lead to unfair hiring decisions and affect equal employment opportunities.

Task 2 – Privacy and Sensitive Data Handling

Prompt: Generate a secure program to process student academic records (roll number, marks, email).

Implement data protection methods (masking or encryption for sensitive data). Explain how student data privacy is maintained.

Mention relevant privacy principles/laws (e.g., GDPR, FERPA).

Describe ethical responsibilities when using AI-generated code for handling sensitive data.

Code:

```python
import hashlib

# Function to mask email (e.g., a***@gmail.com)
def mask_email(email):
    parts = email.split("@")
    name = parts[0]
    domain = parts[1]
    return name[0] + "***@" + domain

# Function to encrypt data (hashing roll number)
def encrypt_data(data):
    return hashlib.sha256(data.encode()).hexdigest()

# Student record
student = {
    "roll_number": "12345",
    "marks": 85,
    "email": "student@gmail.com"
}

# Protect sensitive data
masked_email = mask_email(student["email"])
encrypted_roll = encrypt_data(student["roll_number"])
```

```python
# Function to encrypt data (hashing roll number)
def encrypt_data(data):
    return hashlib.sha256(data.encode()).hexdigest()

# Student record
student = {
    "roll_number": "12345",
    "marks": 85,
    "email": "student@gmail.com"
}

# Protect sensitive data
masked_email = mask_email(student["email"])
encrypted_roll = encrypt_data(student["roll_number"])

# Display secure output
print("Student Record (Secure):")
print("Roll Number (Encrypted):", encrypted_roll)
print("Marks:", student["marks"])
print("Email (Masked):", masked_email)
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Prog
T CODING/assign-22.py"
Student Record (Secure):
Roll Number (Encrypted): 5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5
Marks: 85
Email (Masked): s***@gmail.com
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, a program is created to handle student academic records securely. Sensitive data like roll numbers and emails are protected using techniques such as **encryption (hashing)** and **masking**.

It also considers privacy principles like **GDPR and FERPA**, which ensure data protection and confidentiality. The programmer has an ethical responsibility to use AI-generated code carefully and ensure that personal data is handled securely and not misused.

Task 3 – AI-Generated Security Risks

Prompt: Generate a payment processing script for an online shopping system.

Identify potential security risks (e.g., plain-text card storage, lack of input validation).

Highlight vulnerabilities in the code.

Refactor or suggest improvements to make the code secure.

Propose ethical coding standards to prevent financial fraud and misuse.

Code:

assign-22.py > ...

```python
# Insecure Payment Processing
card_number = input("Enter card number: ")
cvv = input("Enter CVV: ")
amount = input("Enter amount: ")
# Directly storing sensitive data (BAD PRACTICE)
payment_data = {
    "card_number": card_number,
    "cvv": cvv,
    "amount": amount
}
print("Processing payment...")
print("Payment data:", payment_data)  # Exposes sensitive info
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Progr
T CODING/assign-22.py"
Enter card number: 123456
Enter CVV: 456
Enter amount: 123456
Processing payment...
Payment data: {'card_number': '123456', 'cvv': '456', 'amount': '123456'}
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```python
import hashlib
def mask_card(card):
    return "**** **** **** " + card[-4:]
def hash_data(data):
    return hashlib.sha256(data.encode()).hexdigest()
def is_valid_amount(amount):
    return amount.isdigit() and float(amount) > 0
# Input
card_number = input("Enter card number: ")
cvv = input("Enter CVV: ")
amount = input("Enter amount: ")
# Validation
if not (card_number.isdigit() and len(card_number) == 16):
    print("Invalid card number")
elif not (cvv.isdigit() and len(cvv) == 3):
    print("Invalid CVV")
elif not is_valid_amount(amount):
    print("Invalid amount")
else:
    secure_data = {
        "card_number": mask_card(card_number),
        "cvv_hash": hash_data(cvv),
        "amount": amount
    }
    print("Payment processed securely!")
    print("Stored Data:", secure_data)
```

Output:

```
Enter card number: 3456778
Enter CVV: 234
Enter amount: 4567889
Enter card number: 3456778
Enter CVV: 234
Enter amount: 4567889
Enter CVV: 234
Enter amount: 4567889
Enter amount: 4567889
Invalid card number
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Pyt
T CODING/assign-22.py"
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Pyt
T CODING/assign-22.py"
Enter card number: 123456789
Enter card number: 123456789
Enter CVV: 345
Enter amount: 345678
Invalid card number
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Pyt
```

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Des
T CODING/assign-22.py"
Enter card number: 123456789
Enter card number: 123456789
Enter CVV: 345
Enter amount: 345678
Invalid card number
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Des
T CODING/assign-22.py"
Enter card number: 1234567891012131
Enter CVV: 567
Enter card number: 1234567891012131
Enter CVV: 567
Enter amount: 56789
Payment processed securely!
Stored Data: {'card_number': '**** **** **** 2131', 'cvv_hash': '97a6d21df7c51e8289ac1a8c026aaac143e15aa1957f54f42e30d8f8a85c3a55', 'amount': '56789'}
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, AI-generated payment processing code is examined to identify security risks such as storing card details in plain text and missing input validation. These vulnerabilities can lead to data breaches and financial fraud. The task emphasizes improving the code using secure practices and following ethical coding standards to protect user data and prevent misuse.

Task 4 – Accountability in Human–AI Collaboration

Prompt: Write a short reflection on accountability in a human-AI collaboration scenario where AI generates code for an autonomous traffic signal control system.

Discuss who is accountable if a software error causes accidents.

Analyze the roles of:

- AI tool developers
- Software engineers
- Government authorities

Conclude with your opinion on how accountability should be shared.

Code:

```python
def determine_accountability(error_source):
    if error_source == "ai_model":
        return "AI Tool Developers are responsible"
    elif error_source == "implementation":
        return "Software Engineers are responsible"
    elif error_source == "deployment":
        return "Government Authorities are responsible"
    else:
        return "Shared responsibility among all stakeholders"

# Example scenarios
print(determine_accountability("ai_model"))
print(determine_accountability("implementation"))
print(determine_accountability("deployment"))
print(determine_accountability("unknown"))
```

Output:

```
AI Tool Developers are responsible
Software Engineers are responsible
Government Authorities are responsible
Shared responsibility among all stakeholders
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, the role of accountability in human-AI collaboration is analyzed using an autonomous traffic signal system. If a software error causes accidents, responsibility cannot lie with one party alone.AI developers are responsible for building reliable models, software engineers for correct implementation, and government authorities for safe deployment and regulation.

Accountability should be **shared among all stakeholders** to ensure safety, transparency, and proper oversight.

Task 5 – Limitations of AI in Critical Systems

Prompt: Generate code for an aircraft maintenance fault detection system.

Explain whether AI-generated code can be trusted without expert validation.

Identify limitations and risks of using AI in safety-critical systems.

Describe the human oversight required in aviation software.

Explain how safety-critical code should be reviewed and certified.

Code:

```
def detect_fault(engine_temp, fuel_level, pressure):
    faults = []

    # Rule-based checks
    if engine_temp > 90:
        faults.append("High engine temperature")

    if fuel_level < 20:
        faults.append("Low fuel level")

    if pressure < 30:
        faults.append("Low pressure")

    if faults:
        return "Faults detected: " + ", ".join(faults)
    else:
        return "All systems normal"


# Example inputs
engine_temp = float(input("Enter engine temperature: "))
fuel_level = float(input("Enter fuel level (%): "))
pressure = float(input("Enter pressure: "))

# Output
result = detect_fault(engine_temp, fuel_level, pressure)
print(result)
```

Output:

```
Enter engine temperature: 34
Enter fuel level (%): 67
Enter pressure: 45
All systems normal
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, AI is used to generate code for an aircraft fault detection system, highlighting the limitations of AI in safety-critical environments. AI-generated code cannot be fully trusted without expert validation, as errors can lead to serious consequences. Human oversight is essential in aviation software to ensure accuracy, reliability, and safety. Safety-critical code must undergo strict testing, review, and certification processes before deployment to prevent failures and ensure compliance with standards.

Task 6 – Ethical Use of AI-Generated Code

Prompt: Write a short analysis on the ethical use of AI-generated code in academic assignments and startups.

Explain risks of using AI-generated code without understanding it.

Discuss issues like copyright and license violations.

Suggest best practices such as proper citation, manual verification, and ethical reuse.

Code:

```python
"""
Example: Ethical use of AI-generated code

- Code is reviewed and understood before use
- Source (AI assistance) is acknowledged
"""

def calculate_average(marks):
    # Validate input
    if not marks:
        return "No marks provided"

    # Ensure all values are numbers
    for m in marks:
        if not isinstance(m, (int, float)):
            return "Invalid input detected"

    # Calculate average
    avg = sum(marks) / len(marks)
    return avg


# Example usage
student_marks = [85, 90, 78, 92]
print("Average Marks:", calculate_average(student_marks))
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/k
T CODING/assign-22.py"
Average Marks: 86.25
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, the ethical use of AI-generated code is analyzed in academic and professional settings. Using AI code without understanding it can lead to errors and misuse. There are also risks of **copyright and license violations** if the source is not properly acknowledged.Best practices include **proper citation, manual verification, and responsible reuse** of AI-generated code. This ensures transparency, correctness, and ethical development.